Orange Problem–4

Name: Siri Krishna CR

SRN : PES1UG24AM276

SEC: E

Phase 1: Object Storage Foundation

Screenshot 1A — the ./test_objects output showing all tests passing

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ cd ~/ORANGE-4
nano object.c
vboxuser@sirikrishnacram276:~/ORANGE-4$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:112:45: warning: ‘/tmp_XXXXXX’ directive output may be truncated writing 11 bytes into
a region of size between 1 and 512 [-Wformat-truncation=]
  112 |     snprintf(tmp_path, sizeof(tmp_path), "%s/tmp_XXXXXX", shard_dir);
      |                                             ^~~~~~~~~~~
In file included from /usr/include/stdio.h:894,
                 from pes.h:8,
                 from object.c:21:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:71:10: note: ‘__builtin___snprintf_chk’ output betwe
en 12 and 523 bytes into a destination of size 512
   71 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   72 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   73 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vboxuser@sirikrishnacram276:~/ORANGE-4$
```

Screenshot 1B — The sharded directory structure output

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vboxuser@sirikrishnacram276:~/ORANGE-4$
```

Phase 2: Tree Objects

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ find .pes/objects -type f
.pes/objects/62/cc2f2df3ebc217d35f2c4c9fd28d40e72ebddf981e45e1ca92a2071b4a8bdc
.pes/objects/8c/d6c87f76a579f425e4c471d784817b1f4038ea775340d7aa9699577c372fac
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/c2/b3b19dc96c683910f546098cbd45b3e601530e7500e8cb8a8d0544b9daa3c4
.pes/objects/c4/77950a73c7da17c2adcb5ddb924fe37fcd3724a9e7b3cd9c51204c6e975877
.pes/objects/cd/a3217f0666ca8f00dc04996e10cc9bbef28f49cfaff945afb839cdbf372d39
.pes/objects/1e/a9454413ad43f1702f361af1f4b5e29bee9ad61a8b4de5d59d2a7d63835116
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
vboxuser@sirikrishnacram276:~/ORANGE-4$ xxd .pes/objects/62/cc2f2df3ebc217d35f2c4c9fd28d40e72ebd
df981e45e1ca92a2071b4a8bdc | head -20
00000000: 636f 6d6d 6974 2032 3536 0074 7265 6520  commit 256.tree
00000010: 6334 3737 3935 3061 3733 6337 6461 3137  c477950a73c7da17
00000020: 6332 6164 6362 3564 6462 3932 3466 6533  c2adcb5ddb924fe3
00000030: 3766 6364 3337 3234 6139 6537 6233 6364  7fcd3724a9e7b3cd
00000040: 3963 3531 3230 3463 3665 3937 3538 3737  9c51204c6e975877
00000050: 0a70 6172 656e 7420 6364 6133 3231 3766  .parent cda3217f
00000060: 3036 3636 6361 3866 3030 6463 3034 3939  0666ca8f00dc0499
00000070: 3665 3130 6363 3962 6265 6632 3866 3439  6e10cc9bbef28f49
00000080: 6366 6166 6639 3435 6166 6238 3339 6364  cfaff945afb839cd
00000090: 6266 3337 3264 3339 0a61 7574 686f 7220  bf372d39.author
000000a0: 7369 7269 6b72 6973 686e 6120 6372 203c  sirikrishna cr <
000000b0: 5045 5331 5547 3234 414d 3237 363e 2031  PES1UG24AM276> 1
000000c0: 3737 3633 3332 3333 360a 636f 6d6d 6974  776332336.commit
000000d0: 7465 7220 7369 7269 6b72 6973 686e 6120  ter sirikrishna
000000e0: 6372 203c 5045 5331 5547 3234 414d 3237  cr <PES1UG24AM27
000000f0: 363e 2031 3737 3633 3332 3333 360a 0a41  6> 1776332336..A
00000100: 6464 2066 6172 6577 656c 6c              dd farewell
vboxuser@sirikrishnacram276:~/ORANGE-4$
```

Phase 3: The Index (Staging Area)

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  pes.h
  untracked:  tree.c
  untracked:  README.md
  untracked:  index.h
  untracked:  index.c
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  commit.h
  untracked:  test_objects.c
  untracked:  tree.h
  untracked:  test_sequence.sh

304444 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776331591 6 file1.txt
304444 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776331591 6 file2.txt
vboxuser@sirikrishnacram276:~/ORANGE-4$
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ cat .pes/index
304444 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776331764 6 file1.txt
304444 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776331764 6 file2.txt
vboxuser@sirikrishnacram276:~/ORANGE-4$
```

Phase 4: Commits and History

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ make pes
rm -rf .pes
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
./pes log
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
Committed: 1ea9454413ad... Initial commit
Committed: cda3217f0666... Add world
Committed: 62cc2f2df3eb... Add farewell
commit 62cc2f2df3ebc217d35f2c4c9fd28d40e72ebddf981e45e1ca92a2071b4a8bdc
Author: sirikrishna cr <PES1UG24AM276>
Date:   1776332336

    Add farewell

commit cda3217f0666ca8f00dc04996e10cc9bbef28f49cfaff945afb839cdbf372d39
Author: sirikrishna cr <PES1UG24AM276>
Date:   1776332336

    Add world

commit 1ea9454413ad43f1702f361af1f4b5e29bee9ad61a8b4de5d59d2a7d63835116
Author: sirikrishna cr <PES1UG24AM276>
Date:   1776332336

    Initial commit

vboxuser@sirikrishnacram276:~/ORANGE-4$
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/1e/a9454413ad43f1702f361af1f4b5e29bee9ad61a8b4de5d59d2a7d63835116
.pes/objects/62/cc2f2df3ebc217d35f2c4c9fd28d40e72ebddf981e45e1ca92a2071b4a8bdc
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8c/d6c87f76a579f425e4c471d784817b1f4038ea775340d7aa9699577c372fac
.pes/objects/c2/b3b19dc96c683910f546098cbd45b3e601530e7500e8cb8a8d0544b9daa3c4
.pes/objects/c4/77950a73c7da17c2adcb5ddb924fe37fcd3724a9e7b3cd9c51204c6e975877
.pes/objects/cd/a3217f0666ca8f00dc04996e10cc9bbef28f49cfaff945afb839cdbf372d39
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
vboxuser@sirikrishnacram276:~/ORANGE-4$ cat .pes/refs/heads/main
62cc2f2df3ebc217d35f2c4c9fd28d40e72ebddf981e45e1ca92a2071b4a8bdc
vboxuser@sirikrishnacram276:~/ORANGE-4$ cat .pes/HEAD
ref: refs/heads/main
vboxuser@sirikrishnacram276:~/ORANGE-4$
```

Phase 5 & 6: Analysis-Only Questions

Q5.1 — How would you implement pes checkout <branch>?

A branch in PES is basically just a file that stores a commit hash. So when we run pes checkout <branch>, the main idea is to switch from the current commit to the commit that the target branch is pointing to.

To do this, first we need to update the HEAD file so that it now points to the new branch instead of the old one. After that, we read the commit hash stored inside .pes/refs/heads/<branch>. This commit represents the latest snapshot of that branch.

From this commit, we extract the tree object, which represents the full directory structure and file contents. Then we update the working directory by deleting the current tracked files and recreating them using the blobs from the new tree. At the same time, the index also needs to be updated so that it matches this new state.

What makes checkout complex is that it is not just switching a pointer — it actually modifies the entire working directory. It has to carefully replace files, handle directories, and make sure no important data is lost. That’s why it becomes tricky, especially when there are changes in the working directory.

Q5.2 — How to detect dirty working directory conflicts

When switching branches, we must ensure that we are not overwriting any changes the user has made but not committed yet. This is what we call a "dirty working directory".

To detect this, we can compare the current files in the working directory with what is stored in the index. For each tracked file, we read its contents and compute its hash (just like when creating a blob). Then we compare this hash with the one stored in the index.

If the hashes are different, it means the file has been modified after the last commit. Now, if this same file is also different in the target branch, switching would overwrite the user’s changes. In that case, checkout must stop and show an error instead of proceeding.

This method works because the index represents the last committed state, and any difference between the index and the working directory indicates uncommitted changes.

Q5.3 — Detached HEAD

Detached HEAD means that instead of pointing to a branch, the HEAD directly points to a specific commit. In this state, when you make a new commit, it does get created, but it is not attached to any branch.

This means that the commit exists, but no branch is tracking it. If the user later switches to another branch, this commit can become unreachable because nothing points to it anymore.

However, these commits are not immediately lost. The user can still recover them using the commit hash. For example, they can create a new branch from that commit, which will make it reachable again. In real Git, there is also something called a reflog that helps track these temporary commits.

Phase 6: Garbage Collection

Q6.1 — How to find and delete unreachable objects

Over time, the repository accumulates objects like blobs, trees, and commits that are no longer used by any branch. To clean this up, we need to identify which objects are still reachable and which are not.

The idea is to start from all the branch heads and the current HEAD. From each of these commits, we traverse the entire history by following parent commits. For every commit, we also visit its tree, and from the tree we visit all blobs and subtrees. While doing this traversal, we keep track of all visited object hashes using a data structure like a hash set.

After this process, the hash set contains all reachable objects. Then we scan the object store and delete any object that is not present in this set.

For a repository with 100,000 commits and 50 branches, we don't necessarily visit 100,000 × 50 commits. Since branches often share history, many commits overlap. So realistically, we visit around the total number of unique commits and their associated trees and blobs. The hash set helps ensure we don't process the same object multiple times.

Q6.2 — Why garbage collection is dangerous during commit

Running garbage collection at the same time as a commit operation can cause serious problems due to race conditions.

For example, imagine a commit is in progress. It creates a new blob or tree object, but before it finishes updating the references (like the branch pointer), garbage collection runs. Since the new object is not yet referenced by any branch, GC might think it is unreachable and delete it.

Now when the commit completes and tries to reference that object, it no longer exists, leading to corruption or failure.

Real Git avoids this problem by using careful mechanisms. It ensures that objects are written safely before updating references, and it avoids deleting recently created objects. It may also use temporary files and atomic operations to make sure that GC never removes objects that are still in use.